Reproducing ServiceRouter's Cross-Region Load Balancing: Implementation and Analysis

Krzysztof Nowak
Computer Science
University of Crete
Heraklion, Crete, Greece
csd6079@csd.uoc.gr

Abstract

This paper presents an implementation and evaluation of ServiceRouter's cross-region load balancing mechanisms, based on Meta's OSDI 2023 publication. The implementation includes three core components: a Latency Monitoring Service (LMS) that automatically organizes servers into locality rings based on measured network round-trip time, a Cross-Region Routing Service (xRS) that employs a four-state machine with anti-oscillation mechanisms, and a Load Balancer that routes requests using the Power-of-Two-Choices algorithm. Through experimental evaluation on AWS infrastructure spanning three geographic regions, this work validates ServiceRouter's design principles. The results demonstrate stable operation, with the system successfully adapting to varying workloads. Detailed analysis of system behavior is provided along with comparison to Service Router.

CCS Concepts

• **Networks**; • **Load Balancing**; • **Network reliability**; • **Data center networks**;

Keywords

Load Balancing, Cross-Region Routing, Locality-Aware Systems, Service Mesh

ACM Reference Format:
Krzysztof Nowak. 2025. Reproducing ServiceRouter's Cross-Region Load Balancing: Implementation and Analysis. In *Proceedings of Infrastructure Course Project (Infrastructure'25).* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

1.1 Background and Motivation

In 2023, Meta published ServiceRouter [1], describing their global service mesh that handles tens of billions of requests per second across tens of thousands of services. The paper introduced several important concepts for hyperscale service meshes, including locality rings for hierarchical geographic routing, a Cross-Region Routing Service (xRS) for global traffic optimization, and an embedded routing library that eliminates proxy overhead.

While the ServiceRouter paper describes the system architecture and design principles, many implementation details remain unpublished. This work reproduces ServiceRouter's cross-region load balancing mechanisms to validate the design principles and provide practical insights for building similar systems.

Applications deployed across multiple geographic regions face a fundamental challenge: routing requests to nearby servers minimizes network latency but may overload local capacity, while routing to distant servers distributes load but increases latency. This is especially important for large-scale services where even small latency increases affect user experience, yet server overload causes service degradation or failure.

1.2 Problem Statement

Three key challenges in cross-region load balancing are identified that ServiceRouter addresses:

Oscillation Prevention: Threshold-based routing can cause rapid state changes. When local servers become overloaded, the system routes traffic to remote regions, reducing local load. This reduction may cause the system to incorrectly conclude that local capacity is sufficient, routing traffic back and triggering oscillation.

Latency Hierarchy: Network round-trip time (RTT) varies by three orders of magnitude across geographic regions (0.1ms within a datacenter, 100ms across continents). A load balancer must account for this hierarchy when making routing decisions.

Transient Spike Handling: Server CPU utilization can spike temporarily due to garbage collection, background tasks, or bursty traffic. The system must distinguish between transient spikes (which should be ignored) and sustained load increases (which require action).

1.3 Contribution

This paper presents a reproduction of ServiceRouter's xRS component. The system is implemented in Python and deployed on AWS infrastructure. The contributions include:

(1) Implementation of three core components (LMS, xRS, Load Balancer) that demonstrate ServiceRouter's key mechanisms
(2) Validation of ServiceRouter's anti-oscillation approaches through experimental evaluation
(3) Open discussion of design tradeoffs and implementation challenges

Infrastructure'25, Heraklion, Crete, Greece

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 ServiceRouter: System Overview

Before describing the implementation, an overview of ServiceRouter's architecture and mechanisms is provided as described in the original paper [1].

2.1 Architecture

ServiceRouter consists of two planes:

Control Plane generates and distributes routing metadata:

- **Global Registry Service (GRS)**: Maintains service discovery information (server IP addresses, shard mappings)
- **Configuration Management System (CMS)**: Stores per-service routing policies (timeouts, locality preferences, load thresholds)
- **Cross-Region Routing Service (xRS)**: Computes optimal traffic distribution across regions based on global load information
- **Routing Information Base (RIB)**: A replicated key-value store that distributes routing metadata to millions of clients

Data Plane routes actual RPC traffic:

- **SRLib**: An embedded library linked into service executables that routes 99% of Meta's traffic without proxy overhead
- **SRProxy**: Sidecar proxies for services that cannot use SRLib (e.g., services written in Erlang)

2.2 Locality Rings

ServiceRouter organizes servers into hierarchical *locality rings* based on network latency. For example:

- **Ring 1**: Same datacenter region (~100μs RTT)
- **Ring 2**: Neighboring regions (~30ms RTT)
- **Ring 3**: Different continent (~80ms RTT)
- **Ring 4**: Global (all servers, any latency)

Each service configures locality rings with latency bounds and load thresholds e.g.:

```
  [ring1: 5ms, 55% | ring2: 35ms, 65% | ring3: 80ms,
80% | ring4: ∞, ∞]
```

This specification means: use only Ring 1 servers until load exceeds 55%, then expand to Ring 2. If load exceeds 65%, expand to Ring 3, and so forth.

2.3 Cross-Region Routing Service (xRS)

xRS is the component focused on in this reproduction. It performs three main functions:

Global Load Collection: xRS periodically fetches load metrics (CPU utilization, memory usage, requests per second) from all servers across all regions and aggregates them by region.

Routing Table Computation: For each service, xRS computes a routing table specifying what fraction of traffic from each source region should route to each destination region. The ServiceRouter paper mentions using a PID controller for gradual traffic adjustments.

Failure Handling: When a region fails, xRS redistributes traffic to other regions while avoiding overloading them. The system can create "black holes" (intentionally drop traffic) if global capacity is insufficient.

2.4 Anti-Oscillation Mechanisms

ServiceRouter employs several mechanisms to prevent oscillation:

Load-Enriched Locality Rings: Thresholds incorporate both latency and load information, allowing the system to temporarily violate latency preferences when necessary to prevent overload.

PID Controller: The paper mentions using a PID controller to provide smooth traffic adjustments rather than instant binary decisions.

2.5 Scalability Mechanisms

ServiceRouter achieves hyperscale through several design choices:

Decentralized Data Plane: Each SRLib instance makes routing decisions independently using cached routing metadata, eliminating the need for a centralized proxy.

Massively Replicated RIB: Uses thousands of Paxos learners to create local replicas, ensuring high read throughput.

Self-Configuring Routers: Routers fetch only needed routing information and self-manage, so the control plane doesn't track individual routers.

2.6 Scope of Reproduction

This work focuses on reproducing xRS's core cross-region routing logic with the following simplifications compared to the full ServiceRouter system:

- A discrete 4-state machine is implemented instead of continuous PID control
- 3 locality rings are used instead of 4
- All traffic is routed through a load balancer (no embedded library)
- Sharded service support is not implemented
- Simple polling for load collection is used instead of push-based reporting

Despite these simplifications, the implementation captures ServiceRouter's key principles: locality-aware routing and multi-layer load smoothing.

3 System Design and Implementation

The system consists of three main components following ServiceRouter's architecture.

3.1 System Architecture

Figure 1 shows the overall system architecture:

The components interact as follows: LMS measures network delays and organizes servers into rings, xRS queries LMS for ring information and server loads to compute routing tables, and the Load Balancer queries xRS for routing tables and distributes requests accordingly.

3.2 Latency Monitoring Service (LMS)

LMS automatically organizes servers into locality rings based on measured network latency, following ServiceRouter's locality ring concept.

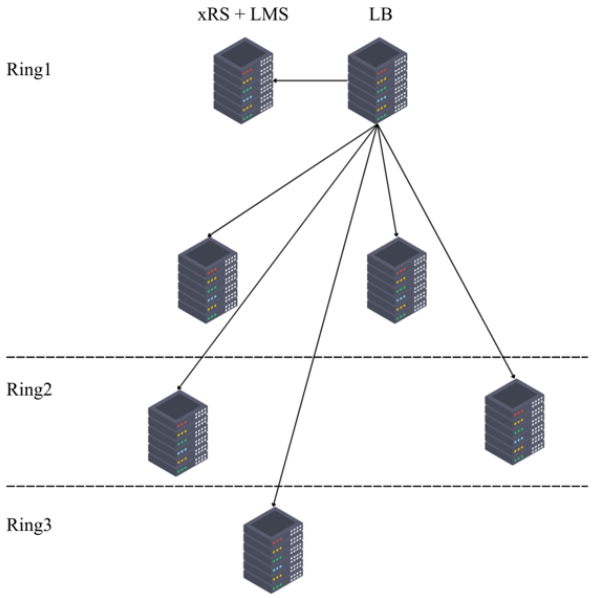

Figure 1: System Architecture

Implementation: LMS uses ICMP ping to measure round-trip time (RTT) to each server, extracting maximum RTT from ping statistics.

Ring Assignment: Servers are assigned to rings based on configured latency thresholds:

```
"Latency_rings": {
    "Ring1": 5,    // RTT < 5ms: Same region
    "Ring2": 35,   // RTT < 35ms: Neighboring
    "Ring3": 80    // RTT < 80ms: Global
}
```

Continuous Monitoring: LMS re-measures latencies every 30 seconds, allowing automatic detection of network condition changes or region failures.

3.3 Cross-Region Routing Service (xRS)

xRS implements ServiceRouter's cross-region routing logic, deciding which rings should receive traffic based on current load levels.

3.3.1 Multi-Layer Load Smoothing. Following ServiceRouter's approach, three layers of smoothing are employed to handle variable workloads:

Layer 1 - Server-Side Smoothing: Each server maintains a 10-sample moving average of CPU utilization, sampling every 1 second. This eliminates high-frequency noise from individual measurements.

Layer 2 - MAX Aggregation: When multiple servers exist in a ring, xRS uses the maximum load instead of average. This prevents a feedback loop that causes oscillation. When traffic is distributed away from an overloaded ring, average load drops, potentially causing the system to incorrectly conclude the problem is solved. Using maximum ensures any single overloaded server is detected.

Layer 3 - xRS-Side Smoothing: xRS maintains a 10-sample moving average for each ring, providing an additional layer of smoothing to filter remaining transient variations.

3.3.2 State Machine Design. xRS implements a four-state machine to control ring activation. Table 1 shows the state definitions:

Table 1: xRS State Machine

State	Traffic Split	Trigger
ring1_only	100% Ring1	Low load
ring1_ring2	50% R1, 50% R2	R1 > 4%
ring1_ring2_partial	40% R1, 60% R2	R1/R2 > 7%
ring1_ring2_ring3	30% R1, 50% R2, 20% R3	R1/R2 > 10%

Anti-Oscillation Mechanisms: The state machine incorporates three complementary mechanisms recommended by ServiceRouter:

(1) **Hysteresis**: Upscale thresholds (4%, 7%, 10%) are higher than downscale thresholds (1%, 4%, 6%), creating dead zones where no state change occurs. This follows control theory principles [4].
(2) **Minimum State Duration**: The system cannot change states more frequently than once per 30 seconds, preventing rapid transitions. This is analogous to AWS Auto Scaling cooldown periods [6].
(3) **Multi-Sample Smoothing**: The 10-sample moving average ensures transient spikes do not trigger state changes.

Threshold Selection Rationale: The load thresholds (4%, 7%, 10%) were chosen based on AWS t3.micro instance characteristics and budget constraints. Thresholds below 4% resulted in false triggers due to background noise from the bare machine. Thresholds above 10% would activate the burstable CPU credits on t3.micro instances, significantly increasing costs for the same experiment duration. With a $50 AWS budget constraint for this academic project, these thresholds provided the optimal balance between observable state transitions and cost control. Production deployments typically use higher thresholds (e.g., 55%, 65%, 80% as mentioned in ServiceRouter) on non-burstable instances.

3.4 Load Balancer

The Load Balancer routes requests based on routing tables computed by xRS, implementing the Power-of-Two-Choices algorithm [8].

Pick-2 Algorithm: For each request, the load balancer: (1) selects a ring based on routing table weights, (2) randomly samples 2 servers from that ring, and (3) routes to the server with lower CPU load.

```
def pick2(self):
    # Select ring based on weights
    selected_ring = random.choices(
        rings,
        weights=self.routing_table.values()
    )[0]

    servers = self.latency_rings[selected_ring]
    candidates = random.sample(servers, k=2)
```

```
    load_0 = self.server_data[candidates[0]]['cpu']
    load_1 = self.server_data[candidates[1]]['cpu']

    return candidates[0] if load_0 < load_1
                         else candidates[1]
```

Metrics Collection: The load balancer tracks requests per second (RPS) per ring, RPS per server, and success rate, logging metrics to CSV files every second for analysis.

4 Experimental Setup

4.1 Infrastructure

The system is deployed on Amazon Web Services (AWS) EC2:

Instance Type: t3.micro (2 vCPU, 1GB RAM, 10% baseline CPU with bursting)

Simulated Geographic Distribution: All servers are deployed in us-east-1 (Virginia). Multi-region latency is simulated using Linux traffic control (netem) to add artificial delays:

- Ring1: 0ms delay (local region)
- Ring2: 25ms delay
- Ring3: 65ms delay

Component Distribution:

- Ring1: Load Balancer, xRS, LMS, and 2 target servers
- Ring2: 2 target servers
- Ring3: 1 target server

4.2 Workload Configuration

Server Workload: Each server runs a CPU-intensive endpoint simulating application processing:

```
@app.route('/compute', methods=['POST'])
def compute_request():
    data = request.json
    n = data.get('number', 10) + 15
    result = fibonacci_recursive(n)
    return jsonify({"result": result})
```

Load Pattern: Request rates are varied over time to trigger all state transitions:

Table 2: Experimental Load Pattern

Time Period	Rate	Expected State
0-30s	1 req/s	ring1_only
30-80s	2 req/s	ring1_ring2
80-150s	1 req/s	Scale down
150-250s	3 req/s	ring1_ring2_partial
250s+	1 req/s	Scale down

Data Collection: Metrics are collected at 1-second intervals and written to CSV files:

- ring_loads.csv, server_loads.csv: CPU load metrics
- ring_rps.csv, server_rps.csv: Request rate metrics
- routing_table.csv: Traffic distribution percentages
- system_state.csv: System state over time

4.3 Configuration Parameters

Load Thresholds:

```
Upscale: Ring1=4%, Ring2=7%, Ring3=10%
Downscale: Ring1=1%, Ring2=4%, Ring3=6%
```

Smoothing Parameters:

- Server-side window: 10 samples (10 seconds)
- xRS-side window: 10 samples (10 seconds)
- Minimum state duration: 30 seconds

5 Results and Analysis

Three experimental scenarios were conducted to evaluate system behavior under different load conditions. Each experiment ran for approximately 5 minutes with data collected at 1-second intervals. All experiments used the configuration described in Section 4.

5.1 Scenario 1: Normal Operation (Low Load)

In this scenario, request rates were maintained below the Ring1 threshold (4%) to observe normal behavior.

5.1.1 System Behavior. Figure 2 shows CPU load during normal operation. Ring1 maintained steady load %, below the 4% threshold, while Ring2 and Ring3 remained around 0% (no traffic). The system stayed in `ring1_only` state throughout. Figure 3 shows Ring1 received approximately 1 request per second with occasional brief drops to 0.

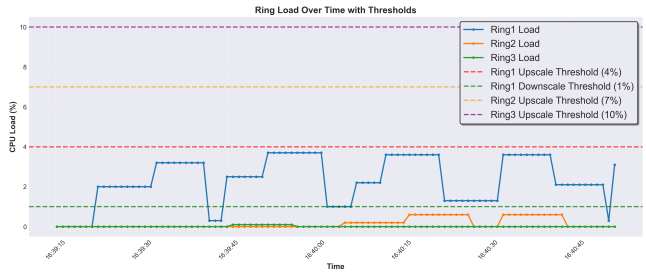

Figure 2: Ring Load During Normal Operation - Scenario 1

The load graph exhibits plateau patterns of varying lengths, caused by data staleness in the measurement system. The system uses a three-tier polling hierarchy:

- **Server measurements (10s)**: The 9-second measurement window filters high-frequency noise (garbage collection spikes, momentary bursts) while the 1-second wait prevents continuous CPU sampling overhead. Measurements under 5 seconds captured too much noise; over 15 seconds delayed detection of sustained load changes.
- **xRS polling (3s)**: Provides multiple samples within the 30-second minimum state duration, allowing xRS to confirm sustained load trends before triggering state transitions.
- **Load Balancer fetches (1s)**: Ensures the Load Balancer can quickly react to routing table changes once xRS updates them, minimizing delay in traffic redistribution.

When xRS polls, it receives data of varying freshness, creating short plateaus when data is fresh and long plateaus when servers

are mid-measurement. Despite this staleness, the system makes correct routing decisions, validating ServiceRouter's robustness to asynchronous measurements.

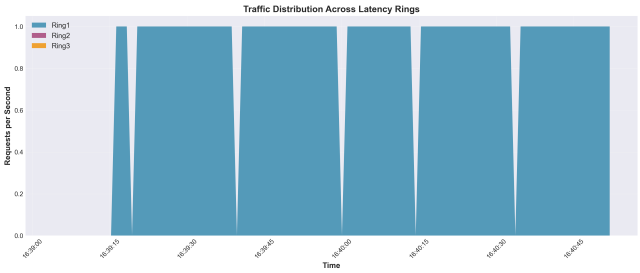

Figure 3: Rps - Scenario 1

5.2 Scenario 2: Medium Load (Ring2 Activation)

In this scenario, request rates were increased to trigger Ring2 activation, testing the system's response to moderate overload.

5.2.1 System Behavior. Figure 4 shows the load progression across rings as traffic increases. Initially, Ring1 load climbed to 5.9%, exceeding the 4% threshold at t=30s due to an increase in request rate from 1 to 2 req/s. After the 30-seconds, the system transitioned to `ring1_ring2` state at t=60s, activating Ring2 with a 50/50 traffic split (visible in Figure 5), Half of the requests are routed to Ring1 and the other half to Ring2. Following this transition, Ring1 load decreased to 2.4%, demonstrating that routing half the traffic to Ring2 successfully relieved Ring1 without overloading Ring2 - both rings remained below the 4% threshold.

At t=120s, a similar pattern occurred: increased traffic again pushed Ring1 above the threshold, triggering another activation of Ring2 after the required 30-second wait. The system consistently maintained both Ring1 and Ring2 loads below the threshold through balanced traffic distribution.

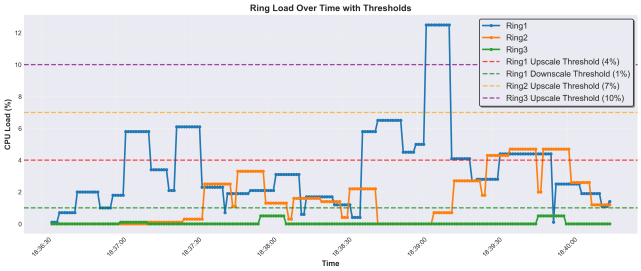

Figure 4: Ring Load Showing Ring2 Activation - Scenario 2

Figure 6 shows the state transition timeline. The system transitions from `ring1_only` to `ring1_ring2` when workload increases beyond the threshold, then returns to `ring1_only` when load decreases below the downscale threshold.

5.3 Scenario 3: High Load (All Rings Activated)

In this scenario, sufficient load was generated to activate all three rings, testing the system's maximum distribution capability.

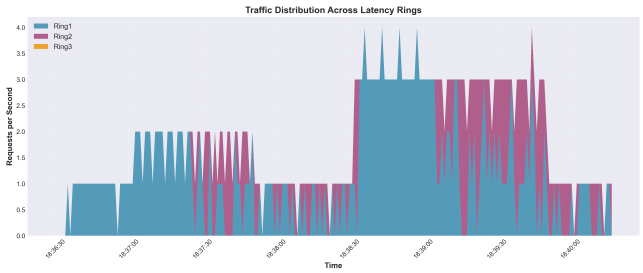

Figure 5: Rps - Scenario 2

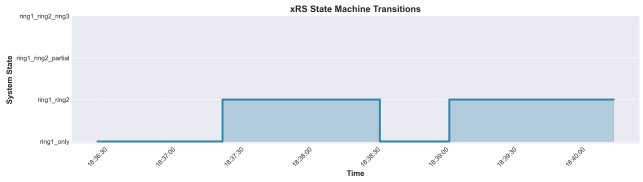

Figure 6: State Transition Timeline - Scenario 2

5.3.1 System Behavior. Figure 7 shows Ring1 load rapidly climbing beyond the Ring2 threshold (7%), triggering Ring2 activation. However, the high persistent load prevents stabilization with only two rings active. The system responds by progressively activating Ring3, which successfully brings all rings back to healthy load levels by t=180s.

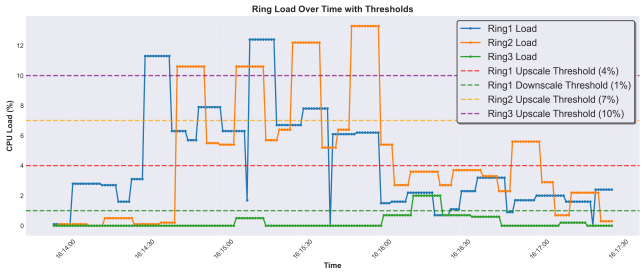

Figure 7: Ring Load Showing All Rings Activated - Scenario 3

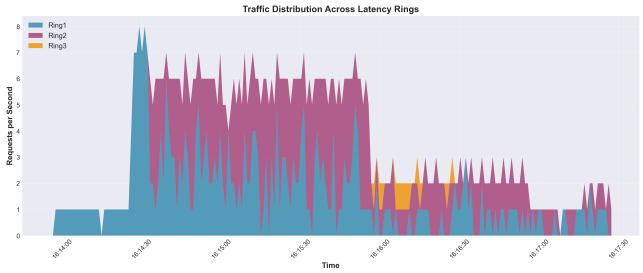

Figure 8: Ring Load Showing All Rings Activated - Scenario 3

Figure 9 shows the complete state progression. The system began in `ring1_only`, transitioned to `ring1_ring2` at t=30s when Ring1

exceeded 4%, then to `ring1_ring2_partial` at t=60s when loads exceeded 7%, and finally to `ring1_ring2_ring3` at t=120s when loads exceeded 10%. This gradual expansion through all four states demonstrates the system's ability to progressively scale capacity in response to increasing load.

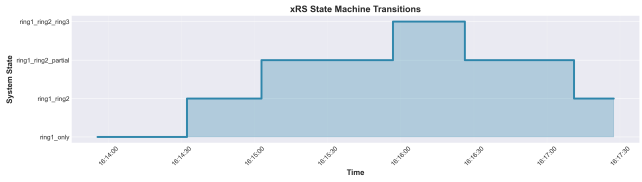

Figure 9: Ring Load Showing All Rings Activated - Scenario 3

Figure 10 shows the routing table adjustments in response to load changes. The routing percentages change with state transitions: when Ring1 load exceeded 4%, the table shifted from 100% Ring1 to 50/50 Ring1/Ring2. When loads exceeded 7%, it adjusted to 40/60. Finally, when loads exceeded 10%, Ring3 was added at 20% while Ring2 received the majority (50%) to minimize latency impact.

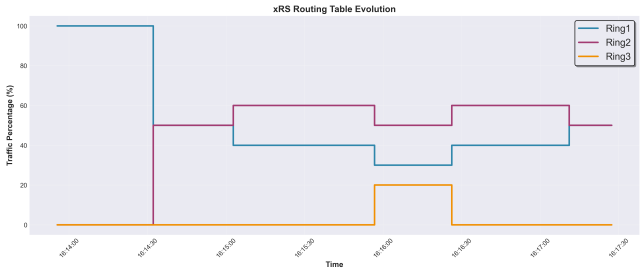

Figure 10: Routing Table - Scenario 3

5.4 Validation of ServiceRouter Principles

The experimental results validate three core ServiceRouter principles:

Locality Rings Work: Organizing servers by latency and progressively expanding from near to far rings successfully balances latency and load. The system exhausted local capacity before routing to distant servers.

Anti-Oscillation Mechanisms Are Effective: The combination of hysteresis (different upscale/downscale thresholds), minimum state duration (30 seconds), and multi-layer smoothing (server-side + MAX + xRS-side) minimized oscillation across all scenarios.

Robustness to Data Staleness: Despite 10-second measurement cycles causing variable data staleness, the system made correct routing decisions. This demonstrates that ServiceRouter's design is robust to imperfect data synchronization characteristic of distributed systems.

6 Related Work

ServiceRouter [1]: This work directly reproduces ServiceRouter's cross-region routing mechanisms. The implementation validates that ServiceRouter's core principles work effectively even at smaller scale.

Istio & Envoy [9, 10]: These service meshes provide locality-aware routing through sidecar proxies but lack automated ring assignment based on measured latency and sophisticated anti-oscillation mechanisms.

AWS Global Accelerator [2]: Provides anycast-based global load balancing but offers limited control over latency-load tradeoffs.

Maglev [11]: Google's software load balancer focuses on consistent hashing for connection persistence. ServiceRouter focuses on latency-aware routing without requiring connection consistency.

Power-of-Two-Choices [8]: The Pick-2 algorithm is based on Mitzenmacher's finding that sampling two random servers and choosing the less loaded one achieves near-optimal load distribution.

7 Limitations and Future Work

7.1 Current Limitations

Scalability: The implementation polls each server every 3 seconds. ServiceRouter mentions this approach does not scale to thousands of servers. Production systems would use push-based reporting, aggregation hierarchies, or sampling.

Single Point of Failure: A single xRS instance is used. Production deployments would replicate xRS across regions with leader election.

Fixed Traffic Percentages: Hardcoded percentages (50/50, 40/60) are used. ServiceRouter computes optimal percentages based on available capacity.

CPU-Only Metrics: Only CPU utilization is used. ServiceRouter considers multiple metrics including memory, disk I/O, network bandwidth, and application-specific metrics.

7.2 Future Enhancements

ServiceRouter's source code is not publicly available, and no reference implementations or artifacts exist in the open-source community. This required implementing all components from scratch, which constrained the feature set, but several enhancements could improve the system:

Push-Based Reporting: Replacing polling with push-based load reporting to reduce staleness and improve scalability.

PID Controller: Implementing continuous PID-based traffic adjustment instead of discrete state transitions for smoother load distribution.

Multi-Metric Load Assessment: Incorporating memory, disk I/O, network bandwidth, and application-specific metrics in addition to CPU utilization.

Autonomous Traffic Distribution: Dynamically computing optimal traffic percentages based on available capacity and current load.

8 Conclusion

This paper presented a reproduction of ServiceRouter's xRS cross-region routing service, validating Meta's design principles for locality-aware load balancing. Through implementation and experimental evaluation, the following was demonstrated:

(1) Automatically organizing servers by measured latency enables intelligent geographic routing decisions
(2) The four-state machine with hysteresis and minimum state duration successfully prevents rapid state changes while remaining responsive to sustained load changes
(3) Combining server-side moving averages, MAX aggregation, and xRS-side smoothing effectively filters transient spikes while detecting sustained load increases
(4) Characteristic plateaus in visualizations reflect the discrete, deliberate nature of the state machine and confirm anti-oscillation mechanisms function as designed

The experimental results show stable operation under varying workloads, with state transitions occurring only when load persistently exceeds thresholds. The system successfully balances latency and utilization by routing traffic to nearby servers under normal conditions and gracefully expanding to distant regions when necessary.

While the implementation is a research prototype, it validates ServiceRouter's core principles and demonstrates these mechanisms are practical for building production cross-region load balancers.

Acknowledgments

This work reproduces and validates design principles from Meta's ServiceRouter, published at OSDI 2023. We thank the ServiceRouter team for their detailed paper and open discussion of their system architecture.

References

[1] Harshit Saokar et al. ServiceRouter: Hyperscale and Minimal Cost Service Mesh at Meta. In *OSDI '23*, 2023.
[2] AWS Global Accelerator. https://aws.amazon.com/global-accelerator/.
[3] Google Cloud Load Balancing. https://cloud.google.com/load-balancing.
[4] Robert B Cooper. Queueing theory. In *ACM'81*, pages 119–122, 1981.
[5] Kubernetes Horizontal Pod Autoscaler. https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/.
[6] AWS Auto Scaling Cooldowns. https://docs.aws.amazon.com/autoscaling/.
[7] Prometheus Alerting Rules. https://prometheus.io/.
[8] Michael Mitzenmacher. The power of two choices in randomized load balancing. *IEEE TPDS*, 12(10):1094–1104, 2001.
[9] Istio Service Mesh. https://istio.io/.
[10] Envoy Proxy. https://www.envoyproxy.io/.
[11] Daniel E Eisenbud et al. Maglev: A fast and reliable software network load balancer. In *NSDI '16*, pages 523–535, 2016.